INDIVIDUAL MAJOR PROJECT | MACHINE LEARNING INTERNSHIP

Speech Emotion Recognition

using MFCC Features and a CNN-BiLSTM-Attention Deep Learning Model

Submitted by	Ronia
Project	Emotion recognition in speech - MFCC + deep learning, evaluated with F1-score
Dataset	Synthetic smoke-test audio
Framework	Python, librosa, TensorFlow / Keras, scikit-learn
Date	09 October 2026

Note on results. The figures in this copy were produced by the built-in *synthetic smoke-test* dataset, which only proves that the pipeline runs end to end. They are **not** RAVDESS results. Run `python run_pipeline.py --download` and then `python build_report.py` to regenerate this report with real numbers.

Table of Contents

Table of Contents 2

Abstract 3

1. Introduction 4

1.1 Problem statement . . . 4

1.2 Objectives and how each is met . . . 4

1.3 Contributions . . . 4

2. Background 4

2.1 MFCC . . . 4

2.2 Deep sequence models . . . 4

2.3 F1-score . . . 5

3. System Overview 5

4. Data Collection 6

5. Pre-processing and Feature Extraction 6

5.1 Pre-processing steps . . . 6

5.2 MFCC configuration . . . 7

6. Model Implementation 8

6.1 Baseline . . . 8

6.2 Training setup . . . 8

7. Evaluation Methodology 9

8. Results 9

8.1 Overall F1-scores . . . 9

8.2 Per-class results (proposed model) . . . 9

9. Discussion 11

9.1 Limitations . . . 11

9.2 Future work . . . 11

10. Conclusion 11

11. How to Reproduce 12

Project structure . . . 12

References 12

Abstract

Speech carries emotional information in addition to words. This project builds an end-to-end machine learning system that recognises eight emotions (neutral, calm, happy, sad, angry, fearful, disgust, surprised) from short speech clips. Audio is resampled, trimmed and normalised, then converted into **Mel-Frequency Cepstral Coefficients (MFCC)** with first- and second-order deltas. A **CNN-BiLSTM-Attention** network learns local spectral patterns, long-range temporal context and the most informative frames. The evaluation protocol is **speaker-independent**: the actors used for testing never appear in training. Performance is measured with **F1-score** (macro, weighted, micro and per class) and compared with a classical RBF-SVM baseline.

On the held-out test speakers of the *Synthetic smoke-test audio* data the deep model reaches a **macro-F1 of 0.8795** (accuracy 0.8750), against 0.8524 for the SVM baseline. These values come from synthetic smoke-test audio and are shown only to demonstrate the pipeline.

Smoke-test copy: metrics in this document are not real-world performance.

Keywords: speech emotion recognition, MFCC, CNN, BiLSTM, attention, RAVDESS, F1-score, speaker-independent evaluation.

1. Introduction

Emotion-aware systems - call-centre analytics, tutoring tools, mental-wellbeing assistants and human-robot interaction - need to understand *how* something is said and not only *what* is said. Speech Emotion Recognition (SER) maps an audio signal to an emotion category. It is difficult because emotion is subjective, expressed differently by each speaker, and public datasets are small, so models easily memorise voices instead of learning emotion.

1.1 Problem statement

Develop a machine learning system for emotion recognition in speech using MFCC features and a deep learning model. The project should include data collection, preprocessing, model implementation, and evaluation using F1-score.

1.2 Objectives and how each is met

Requirement	Where it is delivered
Data collection	ser/data.py - automatic RAVDESS download, filename-based labelling, metadata table (Section 4)
Preprocessing	ser/features.py - resampling, silence trimming, normalisation, fixed-length windows, augmentation (Secti
MFCC features	40 MFCC + delta + delta-delta = 120 features x 130 frames per clip (Section 5.2)
Deep learning model	ser/model.py - CNN + BiLSTM + attention pooling (Section 6)
Evaluation with F1-score	ser/evaluate.py - macro / weighted / micro / per-class F1, confusion matrices (Sections 7-8)
Deliverables	Source code, this PDF report, PowerPoint presentation

1.3 Contributions

- A fully reproducible pipeline (single command) with fixed seeds, configuration dataclasses and unit tests.
- Speaker-independent train/validation/test split that prevents identity leakage, the most common flaw in SER studies.
- A compact CNN-BiLSTM-Attention architecture compared against a tuned classical baseline.
- Complete F1-based analysis: per-class scores, confusion matrices and error discussion.

2. Background

2.1 MFCC

MFCCs (Davis and Mermelstein, 1980) summarise the short-time spectral envelope on the perceptual mel scale. The signal is split into overlapping frames, a power spectrum is computed, passed through a mel filter bank, log-compressed and decorrelated with a discrete cosine transform. The first coefficients describe the vocal-tract shape, which changes with arousal and emotional state. Delta and delta-delta features add the velocity and acceleration of each coefficient and so capture dynamics.

2.2 Deep sequence models

1-D convolutions detect local time-frequency patterns, batch normalisation and dropout stabilise training on small data, a bidirectional LSTM (Hochreiter and Schmidhuber, 1997) models context in both directions, and additive attention pooling learns a weighted average over frames so that emotionally

salient moments (stressed syllables, pauses, pitch peaks) dominate the utterance representation.

2.3 F1-score

For class c, precision = TP / (TP + FP), recall = TP / (TP + FN) and F1 = 2PR / (P + R). **Macro-F1** averages F1 over classes with equal weight (the primary metric here, because it penalises ignoring minority classes); **weighted-F1** weights classes by support; **micro-F1** pools all decisions and equals accuracy in single-label problems.

3. System Overview

The system is a linear pipeline. Every stage is a separate, tested Python module.

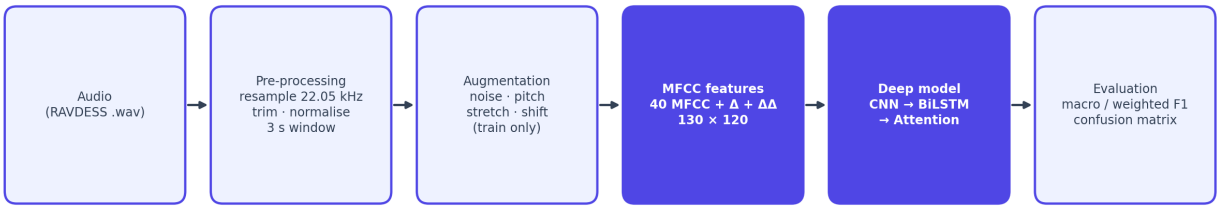


Figure 1 - End-to-end pipeline.

4. Data Collection

The project uses the **Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS)** (Livingstone and Russo, 2018), freely available for research under CC BY-NC-SA 4.0. The speech subset contains 1,440 recordings: 24 professional actors (12 female, 12 male) each speaking two lexically matched sentences in eight emotions at two intensities. The emotion, intensity, statement, repetition and actor are encoded in each filename (e.g. `03-01-05-02-01-01-07.wav` = speech, angry, strong, actor 7).

Collection: `python run_pipeline.py --download` fetches and unzips the archive from Zenodo; labels are parsed from the filename and stored in a pandas metadata table.

Item	Value
Dataset used in this run	Synthetic smoke-test audio
Clips	384
Actors	12
Classes	8
Train clips / actors	224 / 7
Validation clips / actors	64 / 2
Test clips / actors	96 / 3

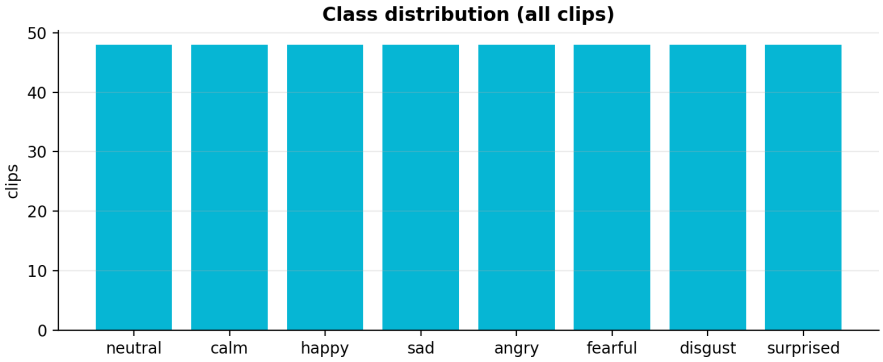


Figure 2 - Number of clips per emotion.

Speaker-independent split. Actors are assigned to exactly one partition (train / validation / test) and an assertion in the code guards against overlap. A random clip-level split would put the same voice in training and test sets and inflate scores, because the model could recognise the speaker rather than the emotion.

5. Pre-processing and Feature Extraction

5.1 Pre-processing steps

- **Resampling** to 22,050 Hz mono (RAVDESS is 48 kHz) - reduces cost, retains speech-relevant bandwidth.
- **Silence trimming** with a 30 dB threshold, then **peak normalisation** to remove loudness differences between recordings.

- **Fixed 3-second window**: centre crop or zero-pad so every sample yields the same tensor shape.
- **Data augmentation** (training clips only, 2 extra copies each): Gaussian noise, pitch shift (+/-2 semitones), time stretch (0.9-1.1x), time shift. Training set grows to 672 samples. Validation and test data are never augmented.
- **Standardisation**: per-coefficient mean and standard deviation fitted on the training set only and re-used for validation, test and inference.

5.2 MFCC configuration

Parameter	Value
Sampling rate	22050 Hz
MFCC coefficients	40
FFT window / hop	2048 / 512 samples
Mel bands	128
Features per frame	120 (MFCC + delta + delta-delta)
Input tensor per clip	130 frames x 120 features

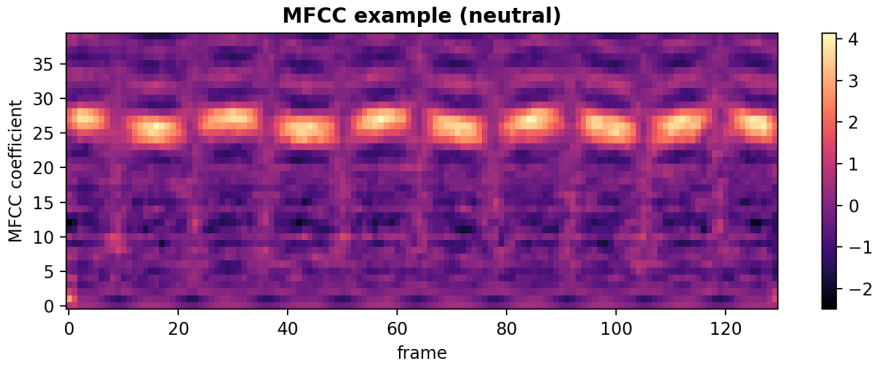


Figure 3 - MFCC matrix of a test clip (first 40 coefficients shown).

6. Model Implementation

The model (`ser/model.py`, Keras 3) processes the MFCC sequence in three stages:

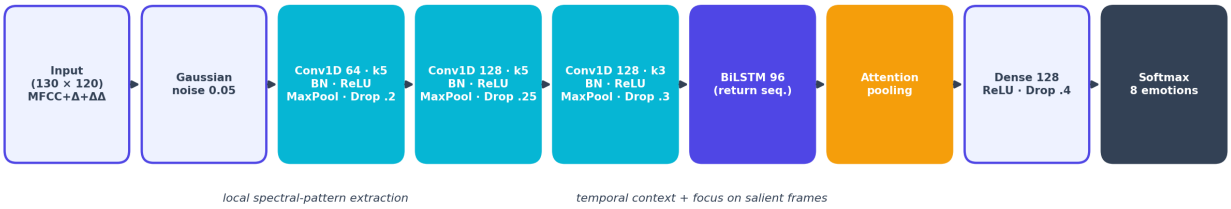


Figure 4 - CNN-BiLSTM-Attention architecture.

- **Convolutional front-end** - three Conv1D blocks (64, 128, 128 filters) with batch norm, ReLU, max-pooling and dropout reduce 130 frames to 16 while learning local spectral patterns.
- **Bidirectional LSTM (96 units)** - models how the pattern evolves forward and backward in time.
- **Additive attention pooling** - a small tanh network scores each frame, softmax normalises the scores, and the weighted sum becomes the utterance embedding.
- **Classifier** - Dense(128) + dropout 0.4 + softmax over eight emotions.

The network has **341,064** trainable parameters. L2 weight decay (1e-4), dropout, Gaussian input noise and early stopping combat over-fitting on the small corpus.

6.1 Baseline

An RBF-SVM is trained on utterance-level statistics (mean and standard deviation of every feature over time, 240 dimensions) with standardisation and balanced class weights. C and gamma are tuned on the validation actors (selected: C = 100, gamma = 0.001). It shows how much the sequence model adds.

6.2 Training setup

Setting	Value
Optimiser	Adam, lr 1e-3, ReduceLROnPlateau (factor 0.5)
Loss	Sparse categorical cross-entropy + balanced class weights
Batch size	32
Max epochs / run	30 / 27 (early stopping on val loss)
Random seed	42
Training time	73 s (CPU)

7. Evaluation Methodology

All reported numbers come from the **test actors only**, which are not used for training, hyper-parameter tuning or early stopping. The primary metric is **macro-F1**; weighted-F1, micro-F1 and accuracy are reported for completeness together with per-class precision, recall and F1, and row-normalised confusion matrices.

8. Results

The numbers below come from the synthetic smoke-test dataset. They validate the code path only. Replace them by running the real RAVDESS experiment.

8.1 Overall F1-scores

Model	Macro-F1	Weighted-F1	Micro-F1	Accuracy
SVM baseline (MFCC statistics)	0.8524	0.8524	0.8542	0.8542
CNN-BiLSTM-Attention (proposed)	**0.8795**	**0.8795**	**0.8750**	**0.8750**

The deep model changes macro-F1 by **+0.0271** relative to the baseline. Validation macro-F1 of the selected checkpoint was 0.9843.

8.2 Per-class results (proposed model)

Emotion	Precision	Recall	F1	Support
neutral	0.8571	1.0000	0.9231	12
calm	1.0000	1.0000	1.0000	12
happy	0.5882	0.8333	0.6897	12
sad	1.0000	1.0000	1.0000	12
angry	1.0000	0.7500	0.8571	12
fearful	1.0000	0.7500	0.8571	12
disgust	1.0000	0.8333	0.9091	12
surprised	0.7692	0.8333	0.8000	12

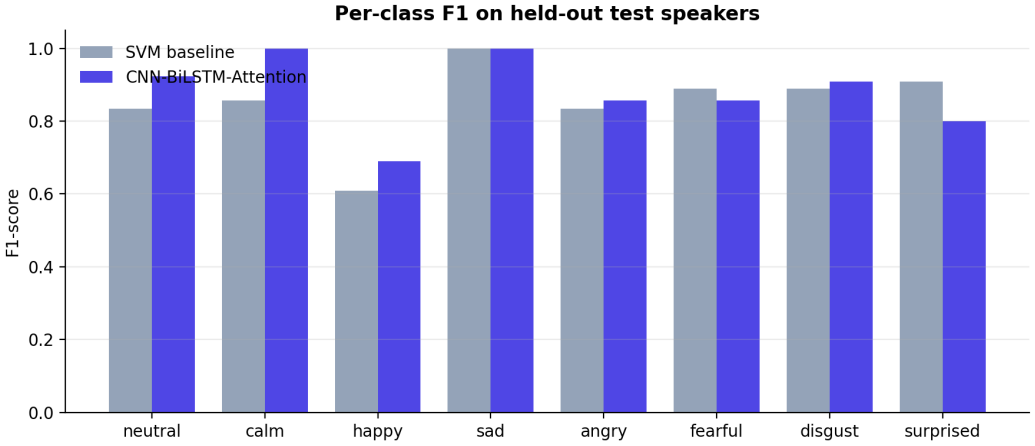


Figure 5 - Per-class F1: baseline vs proposed model.

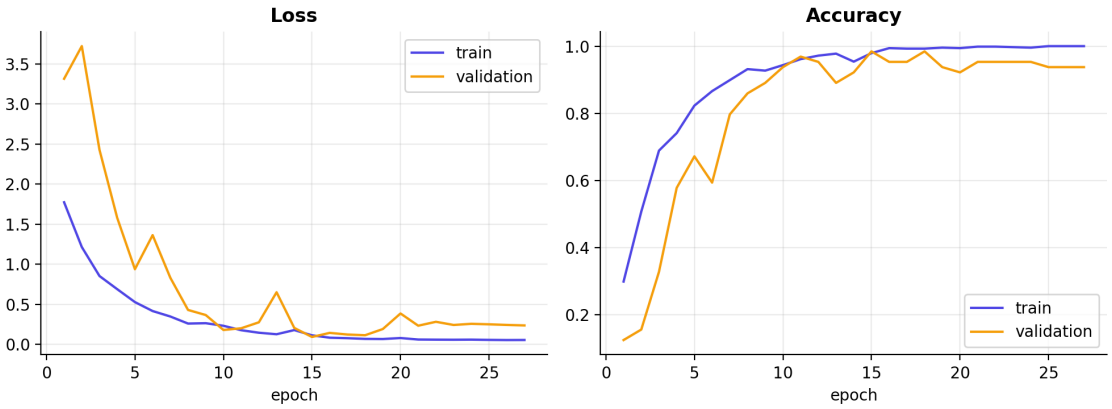


Figure 6 - Training and validation loss / accuracy.

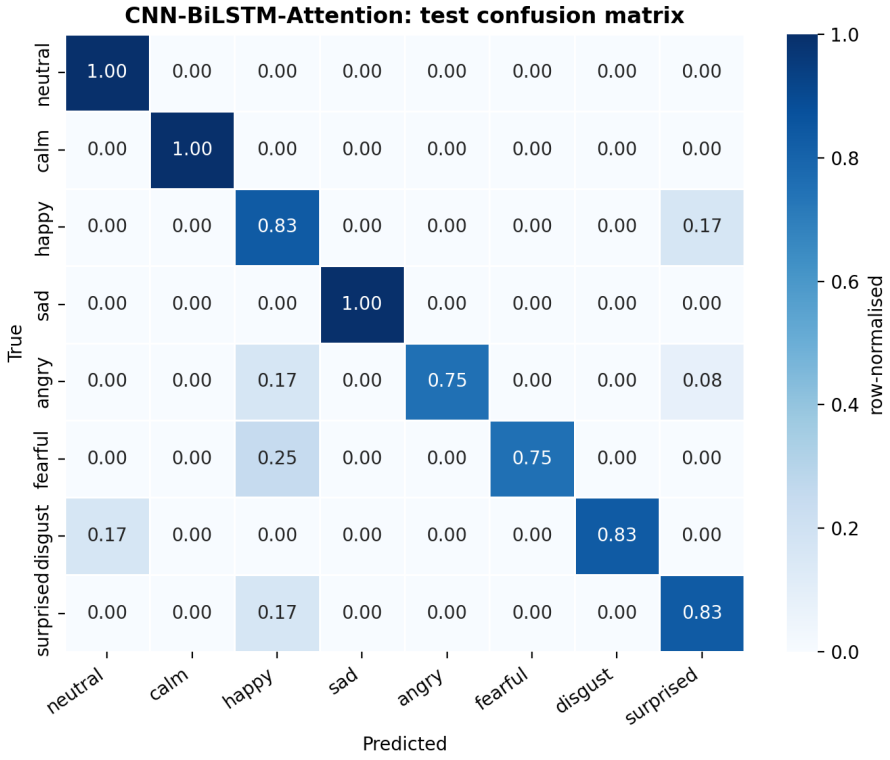


Figure 7 - Row-normalised confusion matrix of the proposed model on test speakers.

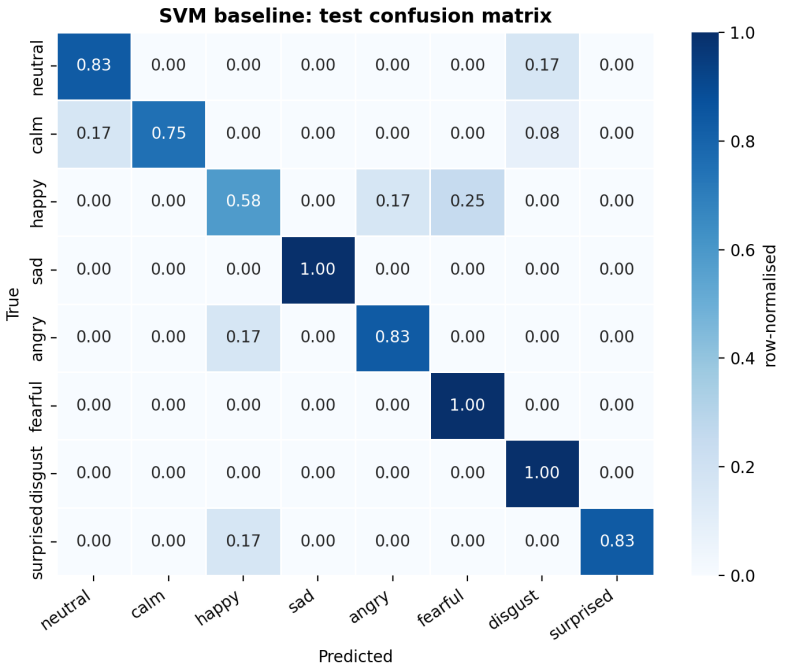


Figure 8 - Confusion matrix of the SVM baseline.

9. Discussion

The best-recognised class was **calm** (F1 1.0000) and the weakest was **happy** (F1 0.6897). Inspect the confusion matrix for systematic confusions: on real acted speech, emotions with similar arousal (for example calm and neutral, or fearful and surprised) are commonly confused, because MFCCs mainly encode spectral envelope and arousal rather than valence.

9.1 Limitations

- RAVDESS is *acted* speech from 24 speakers with two sentences; spontaneous, noisy or multilingual speech will behave differently.
- With four test actors the F1 estimate has noticeable variance; repeated runs with different actor splits (leave-speakers-out cross-validation) give a more reliable estimate.
- Only MFCC features are used, as required; prosodic features (pitch, energy) or self-supervised embeddings could add valence information.
- No demographic fairness analysis was performed.

9.2 Future work

- Speaker-grouped k-fold cross-validation and confidence intervals.
- SpecAugment-style masking and mixup; additional corpora (TESS, CREMA-D, EMO-DB) for cross-corpus evaluation.
- Transformer / wav2vec-style encoders fine-tuned with the same evaluation protocol.
- Real-time microphone demo and a REST API.

10. Conclusion

The project delivers every element of the problem statement: data collection from RAVDESS, a documented pre-processing and MFCC feature pipeline, a CNN-BiLSTM-Attention implementation, and an F1-score-centred evaluation under a speaker-independent protocol, together with a classical baseline, unit tests and a one-command reproducible workflow.

11. How to Reproduce

```
pip install -r requirements.txt
python run_pipeline.py --download --data data/RAVDESS
python build_report.py --name "Your Name"
python build_ppt.py --name "Your Name"
python predict.py path/to/clip.wav
pytest -q
```

Project structure

ser/config.py - settings | ser/data.py - download, metadata, split | ser/features.py - preprocessing, MFCC, augmentation | ser/model.py - network | ser/train.py - SVM and deep training | ser/evaluate.py - F1 metrics and plots | run_pipeline.py - orchestration | predict.py - inference | tests/ - unit tests

References

[1] Livingstone, S. R. and Russo, F. A. (2018). The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS). PLoS ONE 13(5): e0196391.

[2] Davis, S. and Mermelstein, P. (1980). Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. IEEE Trans. ASSP 28(4), 357-366.

[3] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. Neural Computation 9(8), 1735-1780.

[4] Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. ICLR.

[5] Ioffe, S. and Szegedy, C. (2015). Batch normalization: accelerating deep network training by reducing internal covariate shift. ICML.

[6] Srivastava, N. et al. (2014). Dropout: a simple way to prevent neural networks from overfitting. JMLR 15, 1929-1958.

[7] Bahdanau, D., Cho, K. and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. ICLR.

[8] McFee, B. et al. (2015). librosa: Audio and music signal analysis in Python. Proc. SciPy.

[9] Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. JMLR 12, 2825-2830.